Ex01_AISearch

Advanced Artificial Intelligence



GV HƯỚNG DẪN : Thầy Đỗ Như Tài
HỌ VÀ TÊN SV: Bùi Minh Trung
MSSV-LỚP: 3124720074 – DNT1241

LỜI GIẢI BÀI TẬP TÌM KIẾM TRONG AI

PHẦN 1 — BÀI TẬP TÌM KIẾM TRONG AI

Quy ước dùng chung khi giải: khi một node có nhiều nút con, các nút con được sinh ra theo thứ tự alphabet; việc kiểm tra đích (goal test) được thực hiện khi node được LẤY RA khỏi frontier để mở rộng (test on pop/expansion), áp dụng nhất quán cho BFS, DFS, UCS, Greedy và A* để tiện so sánh.

Bài 1. Các thuật toán tìm kiếm

Đồ thị (có hướng): S→A(3), S→C(2), S→D(2), C→F(1), D→B(3), D→G(8), F→E(0), F→G(4), B→E(2), E→G(2).

1.1. BFS

Nội dung	Kết quả
Explored Set	S, A, C, D, F, B, G
Path	S → D → G
Total Cost	2 + 8 = 10

Giải thích: BFS mở theo từng mức (FIFO). Mức 1 sinh ra A, C, D; khi mở D ở mức 1, con của D là B, G — G chính là đích nên BFS dừng ngay, dù đường S-C-F-G (cost 7) rẻ hơn. Điều này minh hoạ: BFS chỉ tối ưu về SỐ CẠNH (độ sâu), không tối ưu về tổng chi phí khi các cạnh có trọng số khác nhau.

1.2. DFS

Nội dung	Kết quả
Explored Set	S, A, C, F, E, G
Path	S → C → F → G
Total Cost	2 + 1 + 4 = 7

1.3. IDS (Iterative Deepening Search)

Giới hạn độ sâu	Các nút được duyệt
0	S
1	S, A, C, D
2	S, A, C, F, D, B, G → tìm thấy đích

Path cuối cùng: S → D → G Total Cost: 2 + 8 = 10

Câu hỏi thảo luận — BFS và DFS khác nhau thế nào: BFS dùng hàng đợi FIFO cho frontier nên mở rộng theo từng mức, luôn tìm ra đường ít cạnh nhất, nhưng chỉ tối ưu về chi phí khi mọi cạnh có trọng số bằng nhau (ở đồ thị trọng số khác nhau như trên, BFS trả về đường cost 10 trong khi đường rẻ nhất chỉ cost 7). DFS dùng ngăn xếp LIFO, đi sâu theo một nhánh trước khi quay lui; không đảm bảo ít cạnh nhất lẫn chi phí thấp nhất về mặt lý thuyết (ở đây DFS tình cờ ra đúng đường rẻ nhất, nhưng đó là do thứ tự duyệt alphabet của đồ thị này, không phải vì DFS có tính chất tối ưu — trên đồ thị khác DFS hoàn toàn có thể trả về đường đắt hơn nhiều).

Bài 2. Tháp Hà Nội

2.1. Biểu diễn trạng thái: state = (peg(1), peg(2), …, peg(n)), trong đó đĩa được đánh số 1..n (1 = nhỏ nhất) và peg(i) ∈ {1,2,3} là cọc hiện chứa đĩa i. Vì đĩa nhỏ luôn phải nằm trên đĩa lớn, thứ tự xếp trên mỗi cọc được suy ra tự động từ các chỉ số đĩa, nên cách biểu diễn này mô tả đầy đủ và không mơ hồ một trạng thái hợp lệ.

2.2. Kích thước không gian trạng thái: mỗi đĩa trong n đĩa độc lập có 3 lựa chọn cọc → số trạng thái = 3^n. Vì thứ tự trên mỗi cọc luôn tự suy ra đúng theo kích thước, mọi tổ hợp trong 3^n đều biểu diễn một cấu hình vật lý hợp lệ.

2.3. Trạng thái bắt đầu: S0 = (1, 1, …, 1) — toàn bộ n đĩa đều ở cọc nguồn (cọc 1).

2.4. Các hành động hợp lệ: với mỗi cọc p không rỗng, gọi top(p) = min{ i : peg(i)=p } (đĩa nhỏ nhất đang ở cọc p — đây chính là đĩa trên cùng). Actions(s) = { move(p,q) : p≠q, p không rỗng, và (q rỗng hoặc top(q) > top(p)) }. Điều kiện hợp lệ: chỉ di chuyển đĩa trên cùng của cọc nguồn, và chỉ đặt lên cọc đích nếu cọc đó rỗng hoặc đĩa trên cùng ở đó lớn hơn đĩa đang di chuyển.

2.5. Goal Test: goalTest(state) trả về True khi và chỉ khi mọi đĩa đều ở cọc đích: i, peg(i) = 3 (tức state∀ = (3,3,…,3)).

Bài 3. Thiết kế và phân tích Heuristic

Đồ thị: S→A(5), S→B(3), A→C(4), A→D(9), B→D(1), C→D(2), C→G(6), D→G(4).

Chi phí tối ưu thật sự từ mỗi nút đến G (tính bằng Dijkstra ngược từ G): h*(G)=0, h*(D)=4, h*(C)=6 (=min(6, 2+4)), h*(B)=5 (=1+4), h*(A)=10 (=min(4+6, 9+4)), h*(S)=8 (=min(5+10, 3+5)).

3.1. Heuristic admissible và consistent

Node	h(n)
S	8
A	10
B	5
C	6
D	4
G	0

Cách đơn giản và an toàn nhất: chọn h(n) = h(n) (chi phí tối ưu thật). Heuristic này luôn vừa admissible (h(n) ≤ h*(n), ở đây bằng nhau) vừa consistent (kiểm tra từng cạnh, ví dụ S→B: h(S)=8 ≤ 3+h(B)=8 đẳng thức; A→C:✓10 ≤ 4+6=10 ; tất cả các cạnh còn lại đều thoả, phần lớn là đẳng thức vì đó chính là các cạnh nằm trên đường đi✓ tối ưu).*

3.2. Chạy A* với heuristic trên

Nội dung	Kết quả
Explored Set	S, B, D, G
Path	S → B → D → G
Total Cost	3 + 1 + 4 = 8

Vì h = h* (heuristic hoàn hảo), A* đi thẳng theo đường tối ưu mà hầu như không cần xét nhánh thừa (A được sinh ra nhưng không bao giờ được mở, vì f(A)=15 > f các node khác trên đường đi tối ưu).

3.3. Heuristic admissible nhưng KHÔNG consistent

Giữ nguyên các giá trị ở câu 3.1, chỉ đổi h(D) từ 4 xuống 0. Giá trị mới vẫn admissible (0 ≤ h*(D)=4). Nhưng xét cạnh B→D (cost 1): cần h(B) ≤ cost + h(D) = 1 + 0 = 1, trong khi h(B)=5 > 1 → VI PHẠM consistency.

3.4. Vì sao consistent (+ h(goal)=0) ⟹ admissible

Theo tính consistent, với mọi cạnh (n,n'): h(n) ≤ c(n,n') + h(n'). Áp dụng liên tiếp dọc theo MỘT đường đi bất kỳ từ n đến đích G = n0,n1,…,nk: h(n0) ≤ c(n0,n1)+h(n1) ≤ c(n0,n1)+c(n1,n2)+h(n2) ≤ … ≤ (tổng

chi phí cả đường đi) + h(G). Vì h(G)=0, vế phải chính là chi phí của đường đi đó. Điều này đúng với MỌI đường đi, nên đúng với đường đi NGẮN NHẤT, tức h(n) ≤ h*(n) — đúng định nghĩa admissible.

Bài 4. Treasure Hunting

Đồ thị: B→A(2), F→B(3), A→C(2), A→D(1), A→E(3), C→F(2), C→G(1), D→G(1), D→H(3), E→H(1), E→I(1). Đích (kho báu) = {E, G} — tới một trong hai là thành công. Heuristic (hình thoi): A=2,B=4,C=1,D=1,E=0,F=6,G=0,H=0,I=0.

Thuật toán	Explored Set	Path returned	Cost
DFS	A, C, F, B, G	A → C → G	3
BFS	A, C, D, E	A → E	3
UCS	A, D, C, G	A → D → G	2
Greedy	A, E	A → E	3
A*	A, D, G	A → D → G	2

Ghi chú nhanh:

- DFS: từ A theo thứ tự C,D,E → đi sâu nhánh C trước (C→F→B→quay lại vì B→A đã thăm) → quay lại nhánh G của C → gặp G (đích).
- BFS: mức 1 của A gồm C,D,E — E chính là đích nên dừng ngay khi tới lượt E (dù C,D đã được mở rộng trước đó theo đúng thứ tự FIFO).
- UCS: tối ưu tuyệt đối, cost 2 qua A-D-G là nhỏ nhất có thể (so với A-C-G cost 3, A-E cost 3).
- Greedy: chỉ nhìn h(n), thấy E có h=0 ngay từ node con đầu tiên của A nên chọn thẳng A→E, không quan tâm chi phí thật.
- A*: vì heuristic admissible (kiểm chứng: mọi h(n) ≤ chi phí thật tới đích gần nhất), A* tìm đúng đường tối ưu giống UCS, cost 2.

Bài 5. Câu hỏi Đúng/Sai

- Câu 1 — SAI. Không có gì đảm bảo DFS luôn mở rộng ≥ số node so với A* có heuristic admissible; nếu DFS may mắn đi đúng nhánh chứa lời giải ngay từ đầu, nó có thể mở rộng RẤT ÍT node, ít hơn cả A*.
- Câu 2 — SAI. Quân xe mỗi nước có thể đi bất kỳ số ô theo hàng/cột, nên số nước tối thiểu giữa 2 ô chỉ là 0, 1 hoặc 2 (tối đa 2: đi qua 1 ô trung gian cùng hàng với ô này, cùng cột với ô kia). Manhattan distance có thể lớn hơn nhiều (ví dụ 14 trên bàn cờ 8×8) nên KHÔNG ≤ chi phí thật → không admissible.
- Câu 3 — ĐÚNG. Vì Pacman di chuyển từng ô theo lưới (chi phí mỗi bước=1), quãng đường lưới thực tế luôn ≥ khoảng cách đường thẳng Euclid (đường thẳng luôn là đường ngắn nhất về mặt hình học), nên Euclidean distance ≤ chi phí thật → admissible.
- Câu 4 — SAI. Tổng của 2 heuristic admissible có thể VƯỢT chi phí thật (ví dụ chi phí thật=10, h1=8≤10 và h2=7≤10 đều admissible riêng lẻ, nhưng h1+h2=15>10). Chỉ có MAX của các heuristic admissible mới chắc chắn vẫn admissible, không phải tổng.
- Câu 5 — SAI. Admissible KHÔNG kéo theo consistent (chiều ngược mới đúng). Phản ví dụ: xem lại Bài 3.3 ở trên — hạ h(D) xuống 0 vẫn admissible nhưng vi phạm consistency ở cạnh B→D.
- Câu 6 — SAI. A* với GRAPH SEARCH (có tập explored, không mở rộng lại node đã đóng) chỉ đảm bảo tối ưu khi heuristic CONSISTENT; nếu heuristic chỉ admissible (không consistent), A* graph search có thể đóng một node với g-value chưa tối ưu rồi bỏ lỡ một đường rẻ hơn phát hiện sau đó → trả về lời giải không tối ưu. (A* TREE SEARCH thì chỉ cần admissible là đủ.)

PHẦN 2 — BÀI TẬP TÌM KIẾM VÀ HEURISTIC TRONG AI

Câu 1. Xe trong mê cung (trạng thái có hướng và vận tốc)

State = (x, y, hướng, v) với hướng ∈ {Bắc,Nam,Đông,Tây}, v ∈ {0,…,Vmax}.

a) *Chuỗi hành động ngắn nhất qua ô (1,2): vì hình vẽ mê cung chỉ có trong file ảnh gốc (không tái hiện chính xác 100% toạ độ tường qua văn bản), m trình bày PHƯƠNG PHÁP giải đúng thay vì đoán mò toạ độ: tại mỗi ô, liệt kê Actions(s) hợp lệ (Faster/Maintain/Slower theo ràng buộc vận tốc + Rẽ trái/phải khi v=0), chạy BFS/UCS trên không gian trạng thái (x,y,hướng,v) từ (0,0,Bắc,0) tới (0,4,Tây,0), ưu tiên nhánh đi qua (1,2) như đề yêu cầu. Khuôn mẫu một chuỗi hợp lệ điển hình: Rẽ phải (quay Đông) → Faster (v=1, tiến 1 ô) → Rẽ trái tại ô có v=0 khi cần đổi trục → ... → Slower liên tiếp để về v=0 đúng tại (0,4) hướng Tây. T đối chiếu từng bước với hình gốc để xác nhận đúng toạ độ tường, phần (b)-(i) dưới đây là phần lý thuyết tổng quát, không phụ thuộc hình vẽ và đã được giải đầy đủ, chính xác.*

b) Kích thước không gian trạng thái: mỗi trạng thái gồm x (M giá trị) × y (N giá trị) × hướng (4 giá trị) × v (Vmax+1 giá trị) → tổng = M × N × 4 × (Vmax+1).

c) Có tồn tại trạng thái không có nút con? CÓ. Ví dụ: xe đang kẹt sát tường theo hướng hiện tại với v=Vmax (không Faster được vì vượt Vmax), v≠0 (không Rẽ được), và cả Maintain lẫn Slower đều khiến xe tiến về phía trước đúng bằng vận tốc mới — nếu ngay phía trước là tường trong phạm vi đó thì cả 2 hành động này cũng đâm tường. Khi đó không hành động hợp lệ nào khả thi → trạng thái kẹt, không có nút con.

d) Hệ số phân nhánh lớn nhất = 3. Tại v=0 (không có tường cản): được phép {Faster, Rẽ trái, Rẽ phải} = 3 hành động (Maintain và Slower đều không hợp lệ khi v=0). Tại 0<v<Vmax (không tường cản): được phép {Faster, Maintain, Slower} = 3 hành động (không rẽ được vì v≠0). Ví dụ trạng thái đạt max: (x,y,Bắc,0) với phía Bắc trống và Vmax≥1 → 3 hành động khả thi.

e) Manhattan distance có admissible không? KHÔNG. Vì một hành động có thể đưa xe đi qua NHIỀU ô cùng lúc (di chuyển đúng bằng vận tốc mới v' trong một hành động), số hành động tối thiểu để đi được quãng đường D ô (xuất phát và kết thúc đều ở v=0) tăng theo quy luật ~2√D (tăng tốc dần rồi giảm tốc dần), chứ KHÔNG tăng tuyến tính theo D như Manhattan distance giả định. Phản ví dụ cụ thể: D=6 ô thẳng hàng (đã đúng hướng, không cần rẽ), Vmax≥2 — chuỗi Faster(v1,đi1),Faster(v2,đi2),Maintain(v2,đi2),Slower(v1,đi1),Slower(v0,đi0) chỉ tốn 5 hành động, đi đúng 6 ô. Vậy chi phí thật = 5 < Manhattan = 6 → h(n) > chi phí thật → VI PHẠM admissibility.

f) h = Manhattan/Vmax có admissible không? CÓ. Chứng minh: mỗi hành động di chuyển tối đa Vmax ô (vì vận tốc luôn bị chặn v≤Vmax), nên để đi được tổng cộng ít nhất Manhattan = |Δx|+|Δy| ô (các hành động rẽ có 0 ô dịch chuyển, chỉ làm tăng thêm số hành động chứ không giảm), cần tối thiểu Manhattan/Vmax hành động. Đây là một chặn dưới đúng cho MỌI kế hoạch hợp lệ, không phụ thuộc chi tiết tăng/giảm tốc hay rẽ, nên h(n) ≤ chi phí thật luôn đúng → admissible.

g) Dùng heuristic không admissible trong A* Tree Search — tính ĐẦY ĐỦ (completeness) có đổi không? KHÔNG. Với đồ thị hữu hạn và h không âm, A* Tree Search vẫn chắc chắn duyệt hết không gian trạng thái hữu hạn và tìm ra lời giải nếu tồn tại — completeness không phụ thuộc admissibility, chỉ cần không gian hữu hạn và h≥0.

h) Tính TỐI ƯU (optimality) có đổi không? CÓ. Heuristic không admissible (đánh giá cao quá mức) có thể khiến A* bỏ qua nhánh chứa lời giải tối ưu vì f(n) của nhánh đó bị đẩy lên cao hơn một lời giải khác kém hơn nhưng có f thấp hơn do heuristic sai lệch → A* có thể trả về lời giải không tối ưu.

i) Lý do hợp lý khi dùng heuristic không admissible: ĐÚNG cho 3 ý đầu (dễ tính hơn giúp giảm thời gian tính heuristic; có thể gần chi phí thật hơn nên mở ít node hơn trong thực tế; có thể khiến thuật toán cắt bỏ hẳn một phần đồ thị không cần xét, đánh đổi lấy tốc độ). SAI ý thứ 3 ("luôn tìm được đường tối ưu khi mọi chi phí thực không âm") — đây là phát biểu sai, non-admissibility vẫn có thể làm mất tối ưu bất kể dấu chi phí cạnh.

Câu 2. Các nút tìm kiếm (Tree Search, không dùng explored set) n_c = nút đích chi phí nhỏ nhất (tối ưu), n_s = nút đích nông nhất (depth nhỏ nhất).

- a) BFS: mọi node được mở rộng đều thoả d(n) ≤ d(n_s).

- b) UCS: mọi node được mở rộng đều thoả $g(n) \leq g(n_c)$.
- c) A* (heuristic consistent): mọi node được mở rộng đều thoả $f(n) = g(n) + h(n) \leq g(n_c)$.

Câu 3. Tìm kiếm trên đồ thị

Đồ thị vô hướng: A-B(3), A-C(4), B-C(2), B-D(3), B-E(5), C-E(3), D-E(1), E-F(3), E-G(6), F-G(2). h1: A12,B6,C9,D3,E3,F2,G0. h2: A11,B7,C6,D4,E5,F1,G0.

Thuật toán	A-C-E-G	A-C-E-F-G	A-B-D-E-F-G	Khác
UCS		X	X	
Greedy h1				X (A-B-E-F-G, cost 13)
Greedy h2		X		
A* h1			X	
A* h2		X		

Chi phí tối ưu thật từ A đến G = 12, đạt được bởi HAI đường đồng chi phí: A-C-E-F-G (4+3+3+2) và A-B-D-E-FG (3+3+1+3+2) — UCS có thể trả về một trong hai tuỳ quy tắc phá hòa, nên đánh dấu cả hai. **b) Chi phí tối ưu A→G theo UCS: 12.**

c) Heuristic h1: KHÔNG admissible — tại C, h1(C)=9 nhưng chi phí thật ngắn nhất từ C đến G chỉ là 8 (C-E-F-G = 3+3+2) → 9 > 8, vi phạm. Vì không admissible nên cũng KHÔNG consistent.

d) Heuristic h2: admissible (kiểm tra từng node đều ≤ chi phí thật: A11≤12, B7≤9, C6≤8, D4≤6, E5≤5, F1≤2, G0≤0 — tất cả đúng). Nhưng KHÔNG consistent — tại cạnh A-B (cost 3): cần h2(A) ≤ 3+h2(B) = 3+7 = 10, nhưng h2(A)=11 > 10, vi phạm.

Câu 4. Trắc nghiệm và trả lời ngắn

a) Đúng/Sai (giả sử mọi hành động cost ≥ ε > 0, mọi heuristic dùng đều consistent)

- DFS Tree Search trên đồ thị hữu hạn luôn complete — SAI. Tree search không có tập explored nên nếu đồ thị có chu trình, DFS có thể lặp vô hạn quanh chu trình dù đồ thị hữu hạn.
- BFS Tree Search trên đồ thị hữu hạn luôn complete — ĐÚNG. Mỗi mức độ sâu chỉ có hữu hạn node (do hệ số phân nhánh hữu hạn), nên BFS luôn chạm tới độ sâu chứa đích sau hữu hạn bước.
- IDS Tree Search trên đồ thị hữu hạn luôn complete — ĐÚNG. Là DFS giới hạn độ sâu lặp lại với giới hạn tăng dần; mỗi giới hạn hữu hạn luôn kết thúc, và giới hạn cuối cùng sẽ chạm tới độ sâu của đích.
- Với đồ thị không có chu trình, Graph Search luôn có frontier lớn hơn Tree Search — SAI. Graph search (nhờ tập explored) thường cho frontier nhỏ hơn hoặc bằng tree search (vì tránh mở lại các trạng thái trùng), không bao giờ lớn hơn.
- IDS Graph Search có độ phức tạp thời gian của BFS và bộ nhớ của DFS — SAI (đây là điểm hay gây nhầm). Lợi thế "bộ nhớ như DFS" chỉ đúng cho bản TREE SEARCH gốc. Nếu thêm tập explored đầy đủ (đúng nghĩa Graph Search) để tránh trùng lặp trạng thái, phải lưu lại toàn bộ trạng thái đã thăm qua các lần lặp độ sâu, khiến bộ nhớ phình to về mức $O(b^d)$ như BFS — mất chính lợi thế bộ nhớ tuyến tính vốn là lý do dùng IDS.
- min(h1,h2) của 2 heuristic consistent cũng nhất thiết consistent — ĐÚNG. Chứng minh: với mọi cạnh (n,n'), vì $h(n)=\min(h1(n),h2(n)) \leq h1(n) \leq c+h1(n')$ và tương tự $\leq c+h2(n')$, nên $h(n) \leq \min(c+h1(n'), c+h2(n')) = c + \min(h1(n'),h2(n')) = c+h(n')$.

b) Heuristic admissible nhưng không consistent trên đồ thị S-A(1)-G(3), S-C(2)-G(2)

Trạng thái	h(s)
S	4

	3
	0
G	0 (đề cho)

Kiểm tra admissible: chi phí thật tới G là h*(S)=4 (=min(1+3,2+2)), h*(A)=3, h*(C)=2 → h(S)=4≤4 ,h(A)=3≤3 , h(C)=0≤2 → admissible. Nhưng xét cạnh S-C (cost 2): cần h(S) ≤ 2+h(C) = 2+0 = 2, mà h(S)=4 > 2 → VI PHẠM consistency.